MOVIEBOOK

DBMS Mini Project — Implementation and Testing Report

Prepared from project notes and reported test outcomes

Database: movie_booking_db | Application: Next.js + MySQL

1. Project Overview

MovieBook is a web-based movie ticket booking application built to demonstrate relational database design and SQL operations through a practical application. Users can browse movies and showtimes, choose seats, confirm a booking, and view booking history. The application uses a Next.js interface and API routes connected to a MySQL database named movie_booking_db.

This report documents the implementation and reported tests in place of database screenshots. It is based on project notes and outcomes shared during development; it is not a claim that every test was independently rerun during report generation.

2. Objectives and Scope

• Design a relational database for users, movies, theatres, screens, seats, shows, bookings, booking-seat relationships, and payments.

• Connect a web interface to MySQL through API endpoints.

• Support seat availability checks and prevent conflicting bookings.

• Use transactions to keep booking, seat allocation, and simulated payment records consistent.

• Demonstrate joins, aggregation, subqueries, views, indexes, stored procedures, triggers, and transaction control.

• Document setup, database design, and testing outcomes.

3. Technology Stack

Layer	Technology / Component
Frontend	Next.js App Router, React, CSS
Backend/API	Next.js API routes
Database	MySQL — movie_booking_db
Database client	mysql2/promise
Database tooling	MySQL Workbench
Payment	Simulated UPI demo record; no real payment gateway

4. Database Design

The reported relational schema contains nine core tables: users, movies, theatres, screens, seats, shows, bookings, booking_seats, and payments. They model users, movie schedules, venue screens, seat inventory, reservations, and payment records.

Table	Purpose
users	Stores user records referenced by bookings.
movies	Stores movie title and descriptive information.
theatres	Stores cinema/theatre details.
screens	Represents screens within theatres.
seats	Stores seat identifiers, screen association, seat number, and seat type.
shows	Connects a movie to a screen, date, time, and ticket price.
bookings	Stores booking owner, show, total amount, and booking status.
booking_seats	Associates a booking and show with each reserved seat.
payments	Stores payment amount, method, status, and associated booking.

A unique constraint on (show_id, seat_id) in booking_seats prevents the same seat from being assigned more than once for a show. Primary and foreign keys connect related records and support referential integrity.

5. Application Workflow

The homepage retrieves movie and show information from the MySQL-backed API.

The user selects a movie and an available show.

The seat API returns seats for the screen and indicates whether each seat is available or occupied.

The user selects seats, subject to a maximum of ten seats per booking.

The booking API validates the show and seat identifiers, verifies that seats belong to the selected show's screen, and checks existing seat assignments.

The server calculates the total using the show price and seat count; it does not trust a client-supplied total.

The server creates the booking, booking-seat rows, and a simulated payment record inside a MySQL transaction.

On success, the transaction commits and the confirmation is displayed. The booking appears in My Bookings after data is loaded from the database.

6. SQL Concepts Demonstrated

SQL concept	How it is used
DDL and DML	Create database structures and insert, update, and query data.
Multi-table JOIN	Combine shows with movies, screens, and theatres; join bookings with related records.
GROUP BY and aggregates	Summarize booking counts and totals by movie or other grouping.
Subqueries	Queries that depend on results from another query.
Views	Reusable query definitions, including booking detail/summary views.
Indexes	Support lookups and enforce uniqueness where defined.
Stored procedure	GetShowsByDate was reported as created and returning rows.
Triggers	check_booking_amount and validate_show_price were reported as existing; see verification note.
Transactions	COMMIT and ROLLBACK demonstrate atomic changes.
Row locking	SELECT ... FOR UPDATE and conflict checks protect concurrent booking attempts.

Example reporting query:

```
SELECT
    m.title AS movie,
    COUNT(b.booking_id) AS total_bookings,
    SUM(b.total_amount) AS total_revenue
FROM movies m
LEFT JOIN shows s ON s.movie_id = m.movie_id
LEFT JOIN bookings b ON b.show_id = s.show_id
GROUP BY m.movie_id, m.title;
```

This example demonstrates JOIN, GROUP BY, COUNT, and SUM. For formal revenue reporting, filter to the booking and payment statuses required by the report.

7. Booking API and Data Integrity

• POST /api/bookings accepts the selected show ID and seat IDs; the current demo resolves a default user from the database.

• The server validates the number and uniqueness of selected seats and checks that they belong to the screen for the selected show.

• The server calculates the total from the show price and number of seats.

• Booking, booking_seats, and payments inserts occur in one transaction, with rollback on failure.

• The unique key on (show_id, seat_id) provides an additional duplicate-reservation safeguard.

• Payment is simulated and marked as a demo transaction; no real payment gateway is integrated.

A defect was identified: a cancelled seed booking still had a row in booking_seats, while the original availability logic treated only confirmed bookings as occupied. Availability and conflict checks were changed to treat seats already recorded in booking_seats as unavailable. Duplicate-key errors are handled as a conflict instead of a generic server error.

8. Testing and Verification Results

Test / check	Reported result	Notes
MySQL connection endpoint	Passed	/api/db-test returned a successful connection.
Movie API	Passed	/api/movies returned movie records.
Show API	Passed	/api/shows returned show and venue details.
Seat availability API	Passed	Seat statuses were checked against booking-seat records.
Booking conflict test	Passed	Occupied seat request returned HTTP 409 Conflict.
Successful API booking	Passed	Booking #39 reported created for show 9, seat ID 33, total ■150.
Persistence check	Passed	Booking #39 reported in bookings, booking_seats, and payments.
Repeated booking attempt	Passed	Seat ID 33 for the same show was rejected with HTTP 409.
Website booking flow	Passed	Shared screenshot showed booking #40 and the My Bookings entry.
Later website booking	Passed	Shared screenshot showed booking #41: Sky Warriors, seat B4, ■150.
Production build	Passed (reported)	npm run build completed successfully.
Lint	Passed (reported)	npm run lint reported zero errors and warnings.

Test / check	Reported result	Notes
Stored procedure	Reported passed	GetShowsByDate reportedly returned nine rows.
Transaction rollback	Reported passed	A temporary amount change was rolled back; amount reported restored to ■300.
Trigger behavior	Not fully verified	Trigger names were reported present; full behavior test was not confirmed.

These results summarize reported development tests. The user-shared screenshots support successful booking confirmation and booking-history display. Build, lint, and API test results were reported by the development process rather than independently rerun while preparing this report.

9. Limitations

• The payment step is simulated and does not process real payments.

• The demo resolves a default user from the users table rather than demonstrating a complete authenticated customer session.

• Automated browser testing was blocked by an external Playwright driver download issue; the development report states that API requests were tested separately.

• Exact definitions and behavior of every view, index, and trigger should be rechecked if live evidence is required.

• The assignment rubric should be checked for a second database paradigm requirement. MongoDB dependencies were removed; the implementation described here uses MySQL.

10. Setup and Execution

Install Node.js and MySQL, then open the project directory.

Create/select movie_booking_db and run the supplied SQL initialization script (movie_booking_project.sql) in MySQL Workbench.

Create .env.local in the project root with DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, and DB_NAME. Never commit this file or expose credentials.

Install dependencies with npm install.

Start the app with npm run dev.

Open http://localhost:3000 and test browsing, seat selection, booking confirmation, and My Bookings.

Use README.md and the docs/ directory for detailed setup, architecture, data dictionary, SQL examples, and testing notes.

11. Conclusion

MovieBook demonstrates how a relational database supports a practical movie-ticket booking workflow. The project combines a web interface with MySQL-backed APIs and applies relational modelling, joins, aggregation, reusable database objects, transaction control, and conflict prevention. Reported tests show successful bookings being stored and displayed in booking history while duplicate seat reservations are rejected. It should be presented as an academic prototype with simulated payment, not as a production payment platform.

Appendix A. Reported SQL objects and files

• Database: movie_booking_db.

• Core tables: users, movies, theatres, screens, seats, shows, bookings, booking_seats, payments.

• Reported procedure: GetShowsByDate.

• Reported triggers: check_booking_amount, validate_show_price.

• booking_details was reported as created and returning 20 records. Other documented view names included movie_show_details, booking_summary, and payment_report; verify exact availability before claiming all are present.

• SQL files previously referenced: movie_booking_project.sql, movie_booking_testing.sql, and numbered scripts for views, indexes, procedures, and trigger tests.

Appendix B — Application Screenshots

The following screenshots were supplied during project testing and document the visible application workflow.

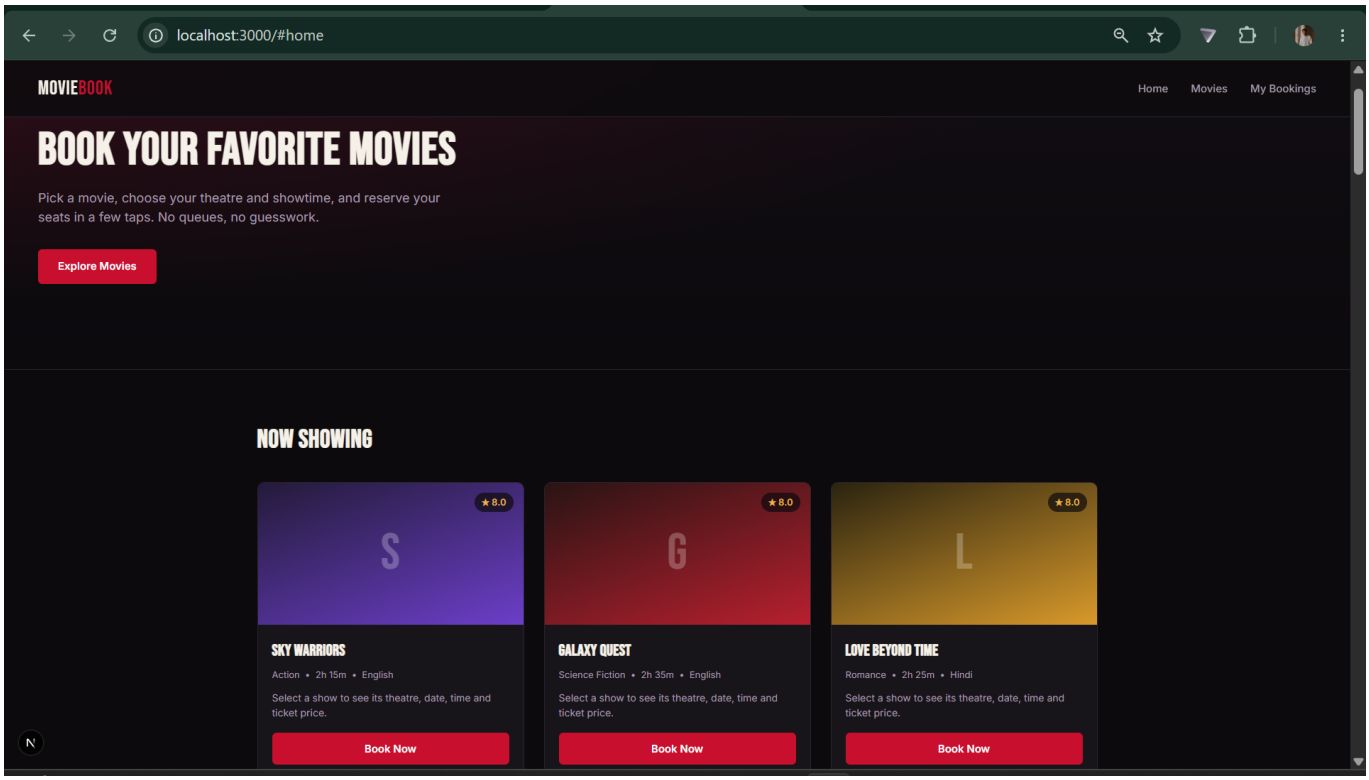


Figure 1. MovieBook homepage and movie listings.

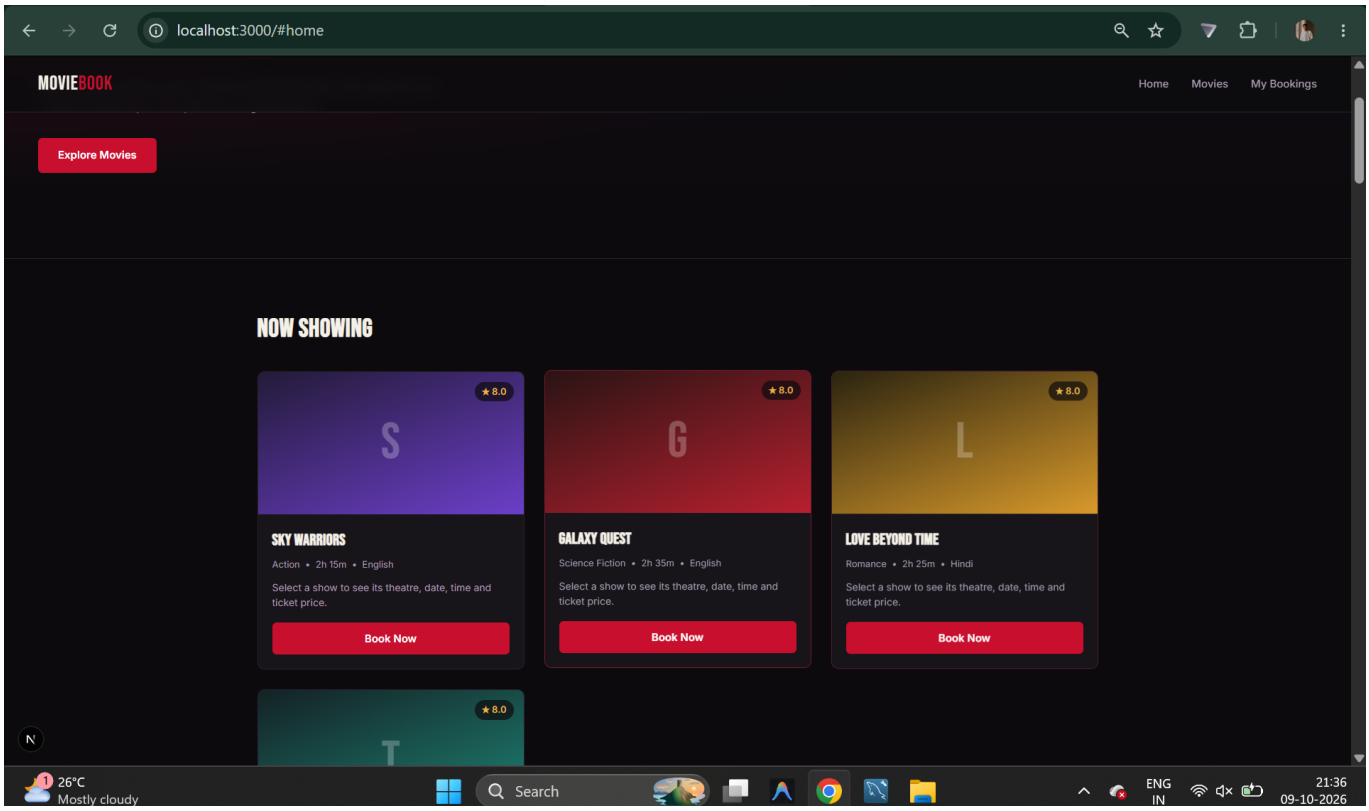


Figure 2. Movie listings and available movie cards.

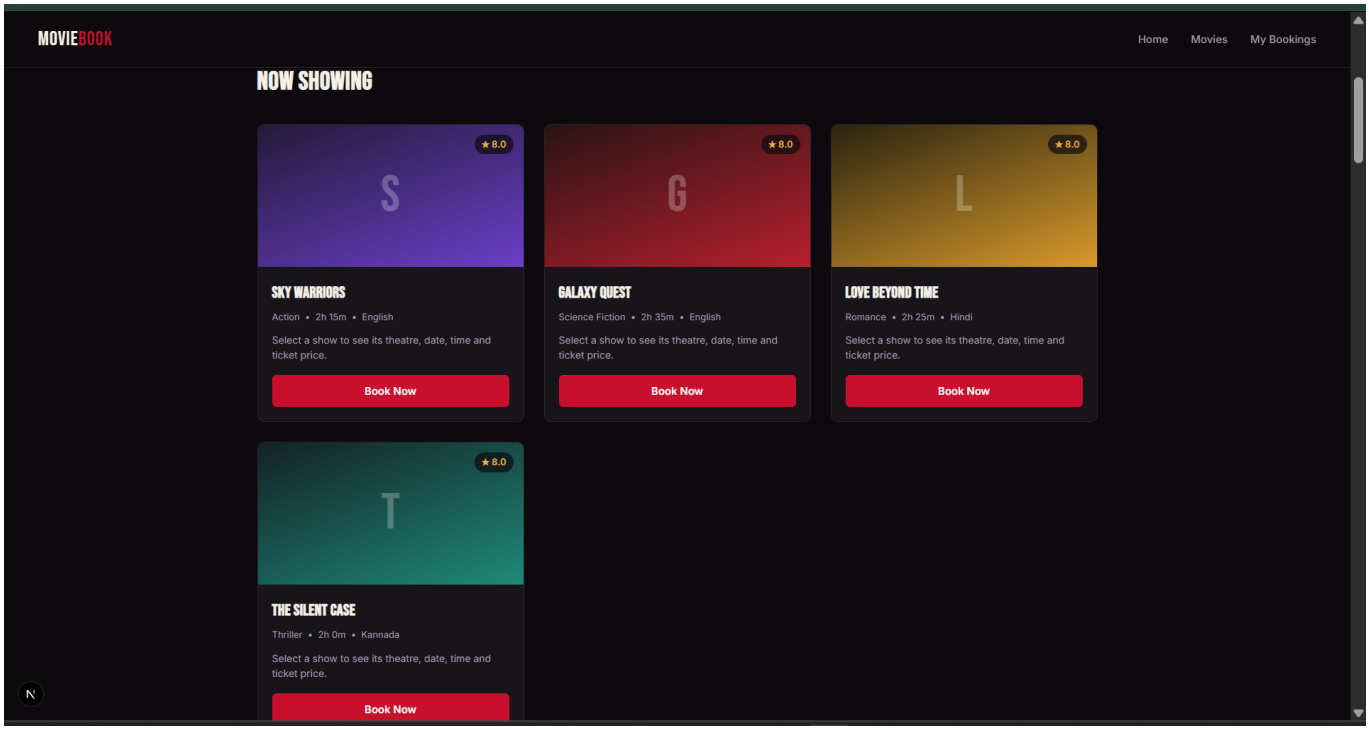


Figure 3. Movie cards and booking interface view.

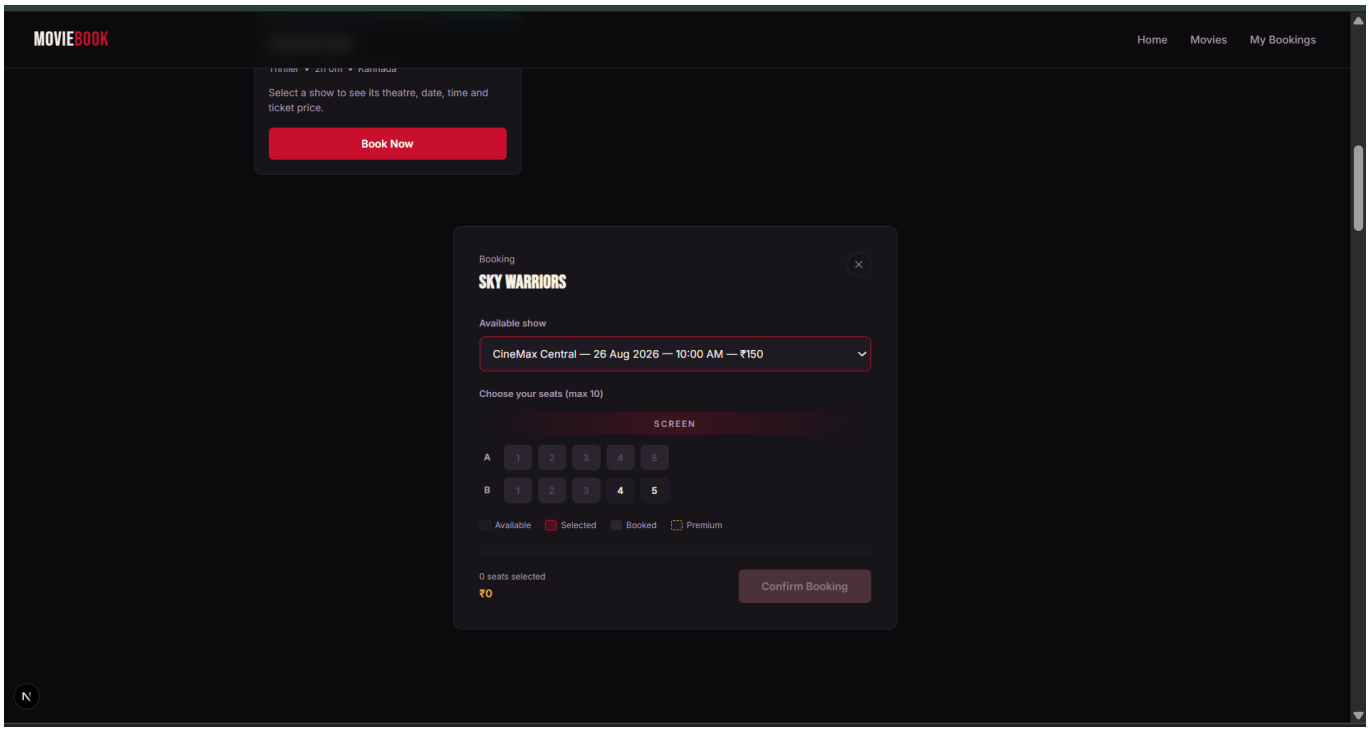


Figure 4. Seat-selection interface with show details and seat status legend.

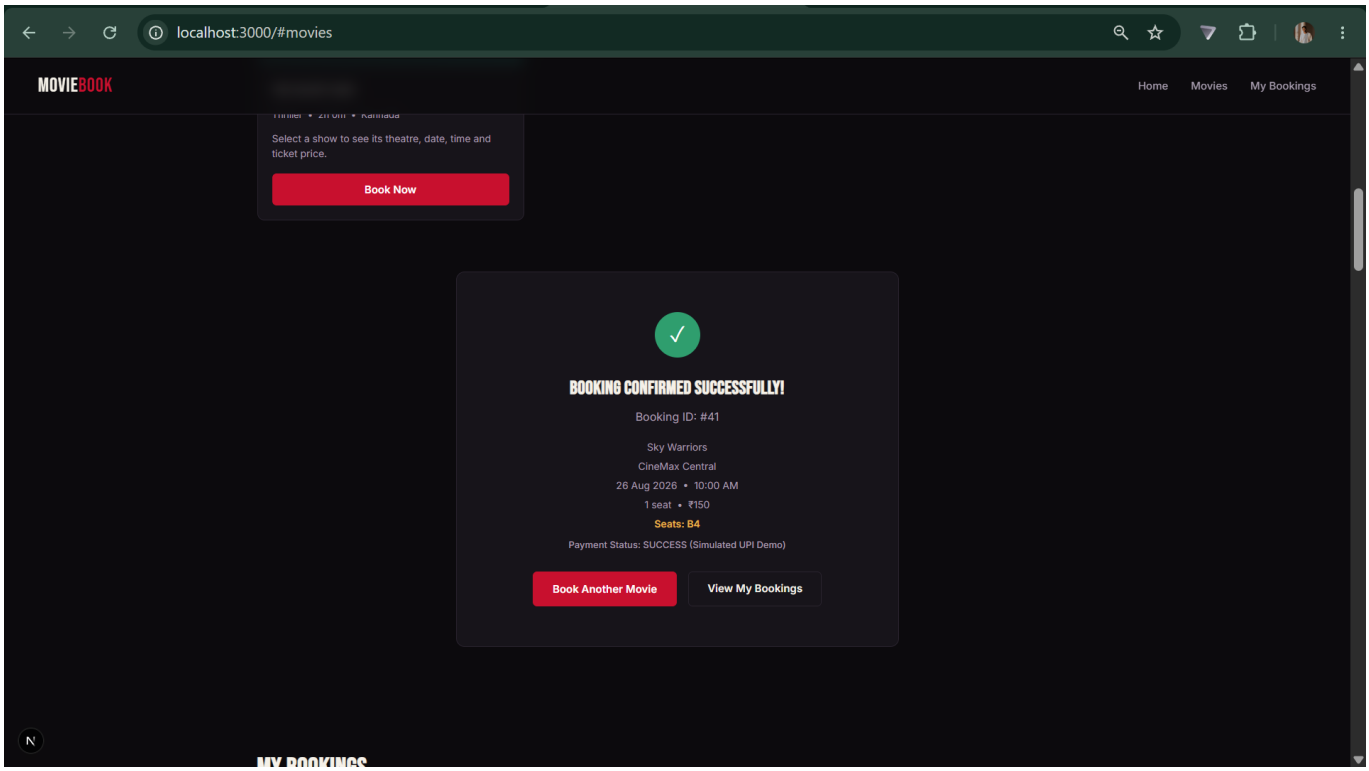


Figure 5. Successful booking confirmation for booking #41.

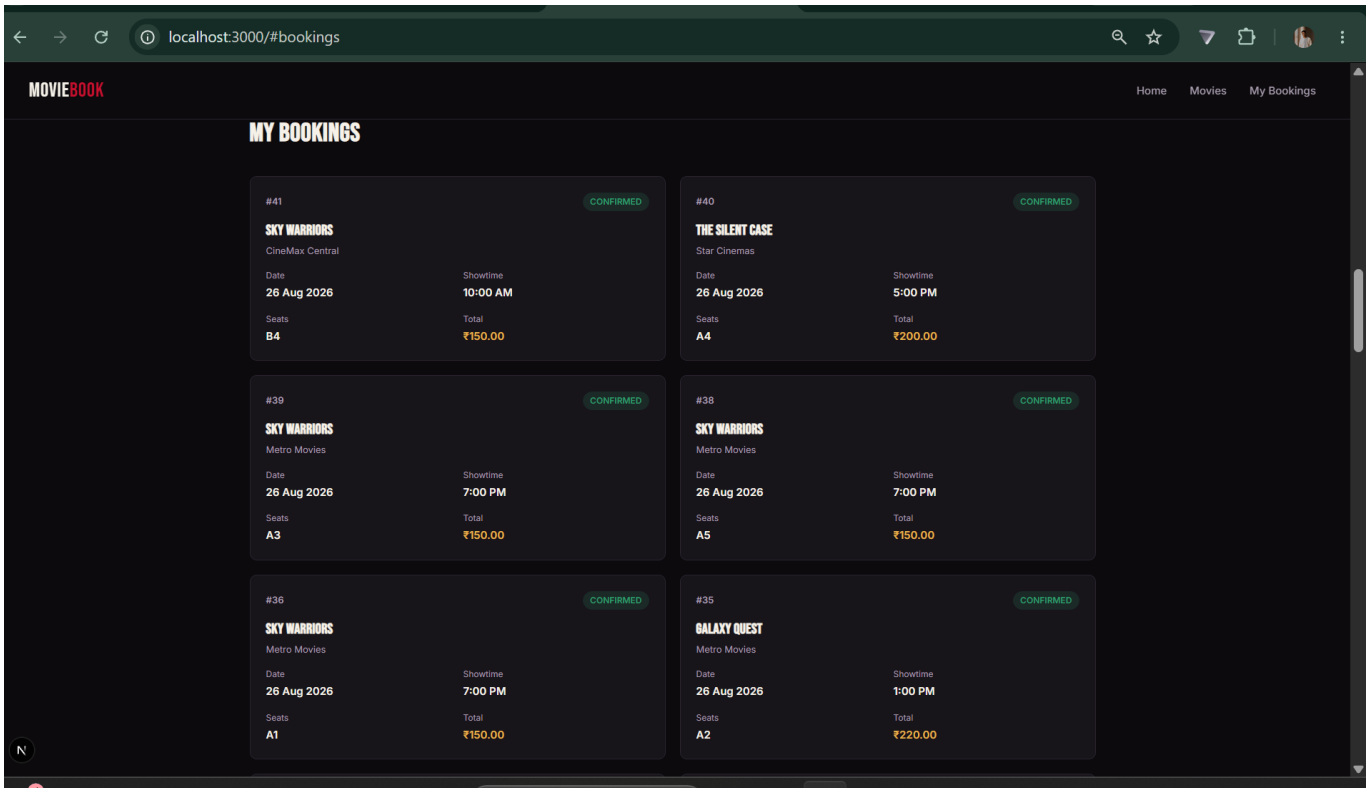
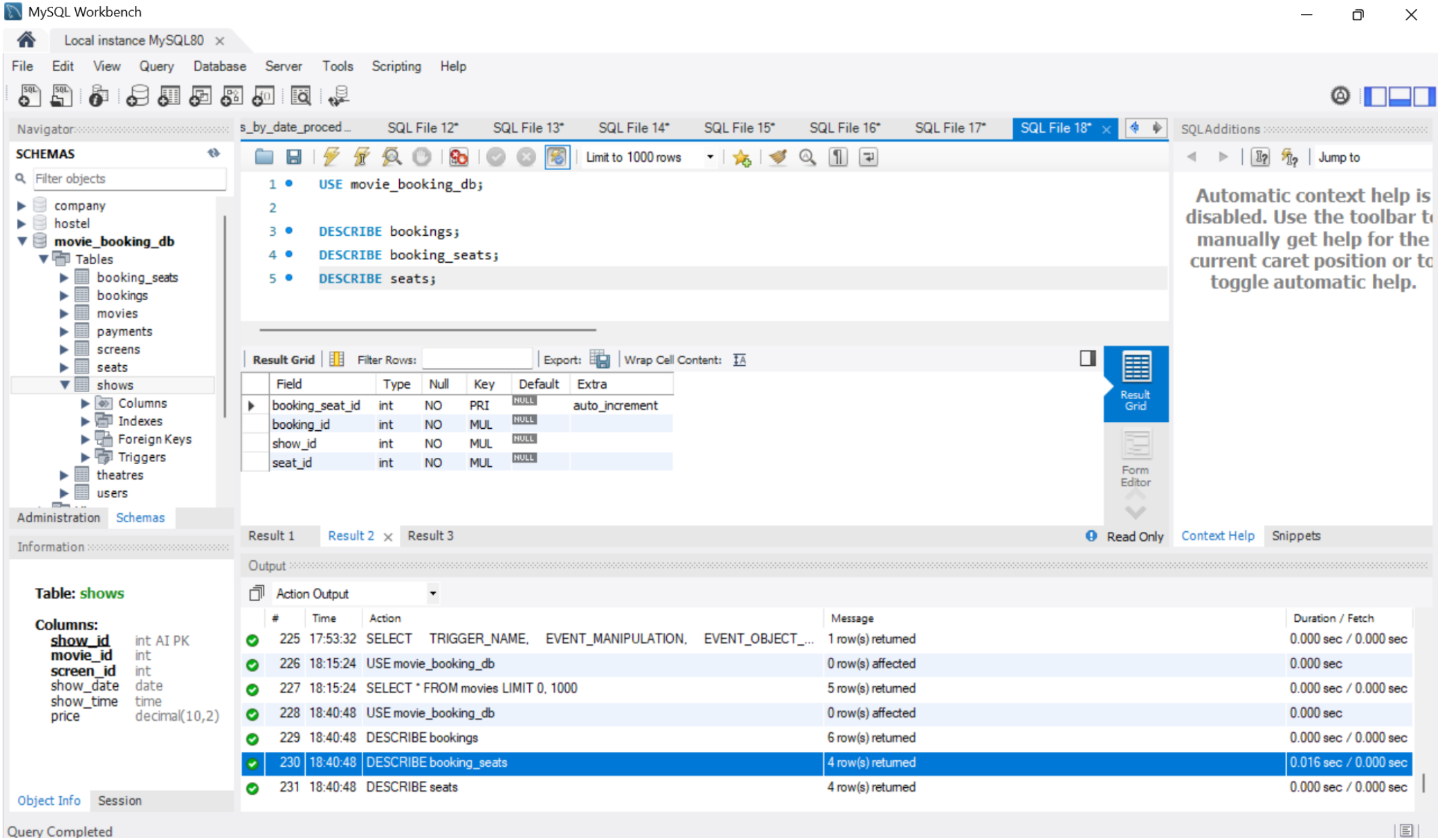


Figure 6. My Bookings page showing saved booking records.

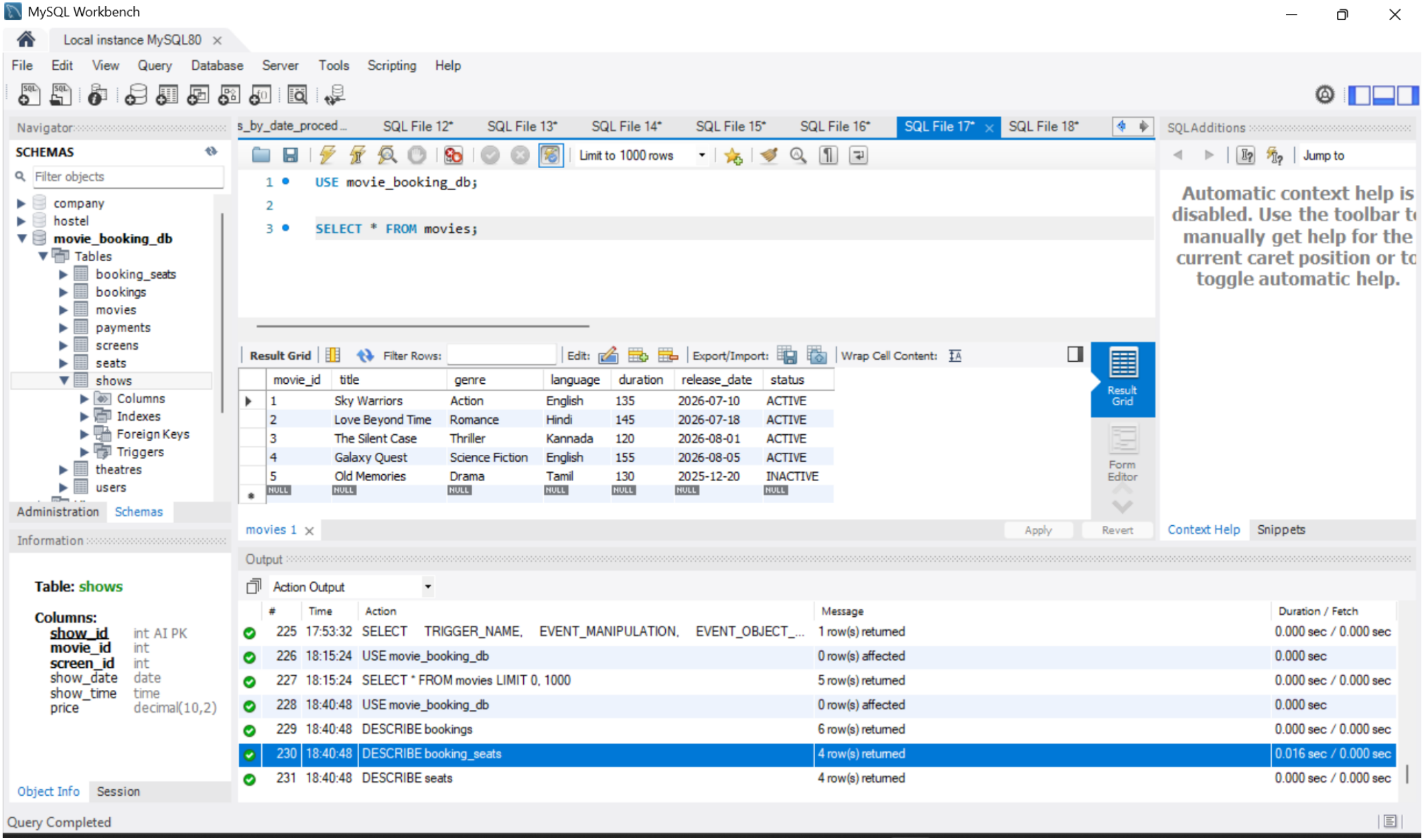
MySQL Workbench Evidence 1

Database schema and table structure (DESCRIBE statements)



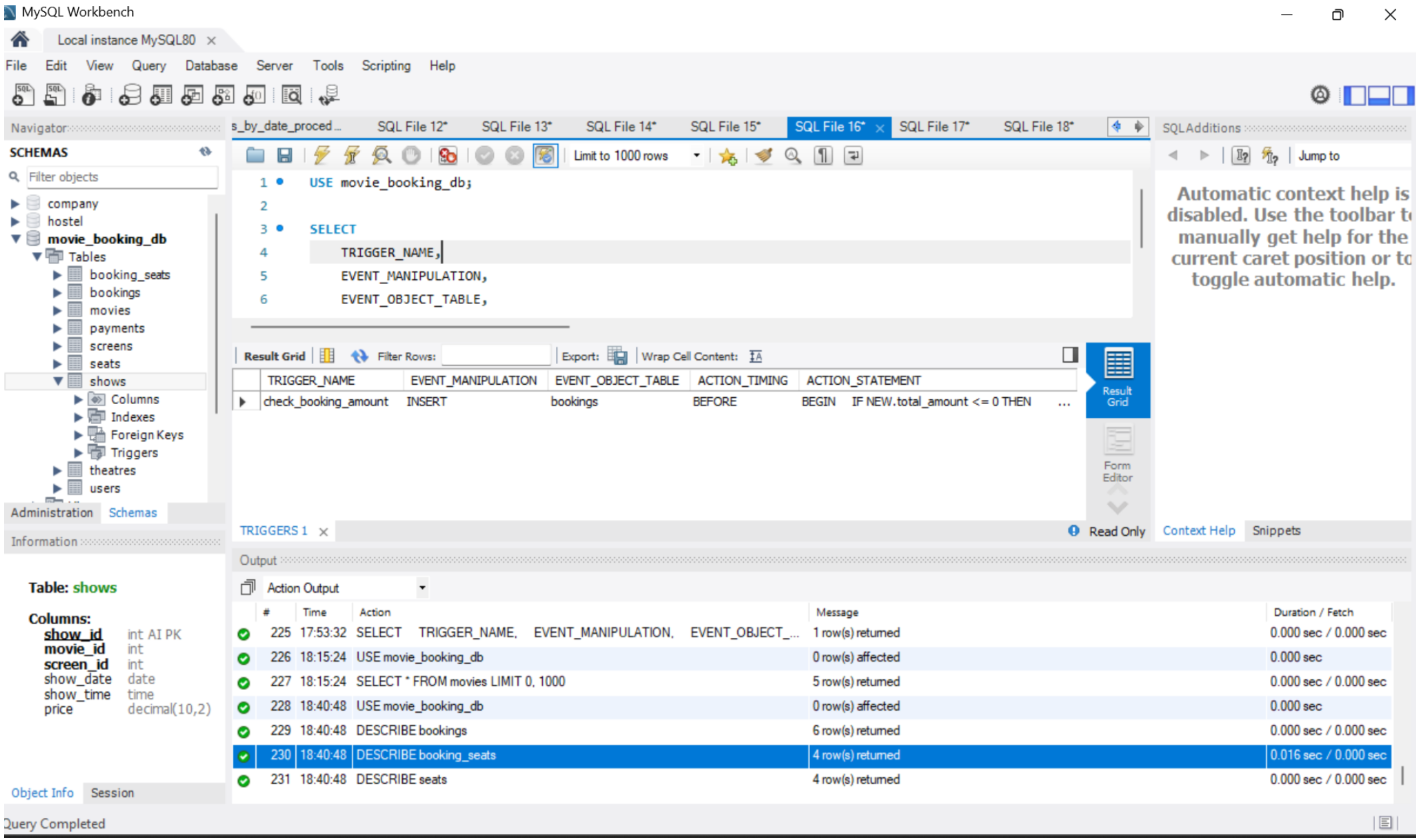
MySQL Workbench Evidence 2

Movie records stored in movie_booking_db



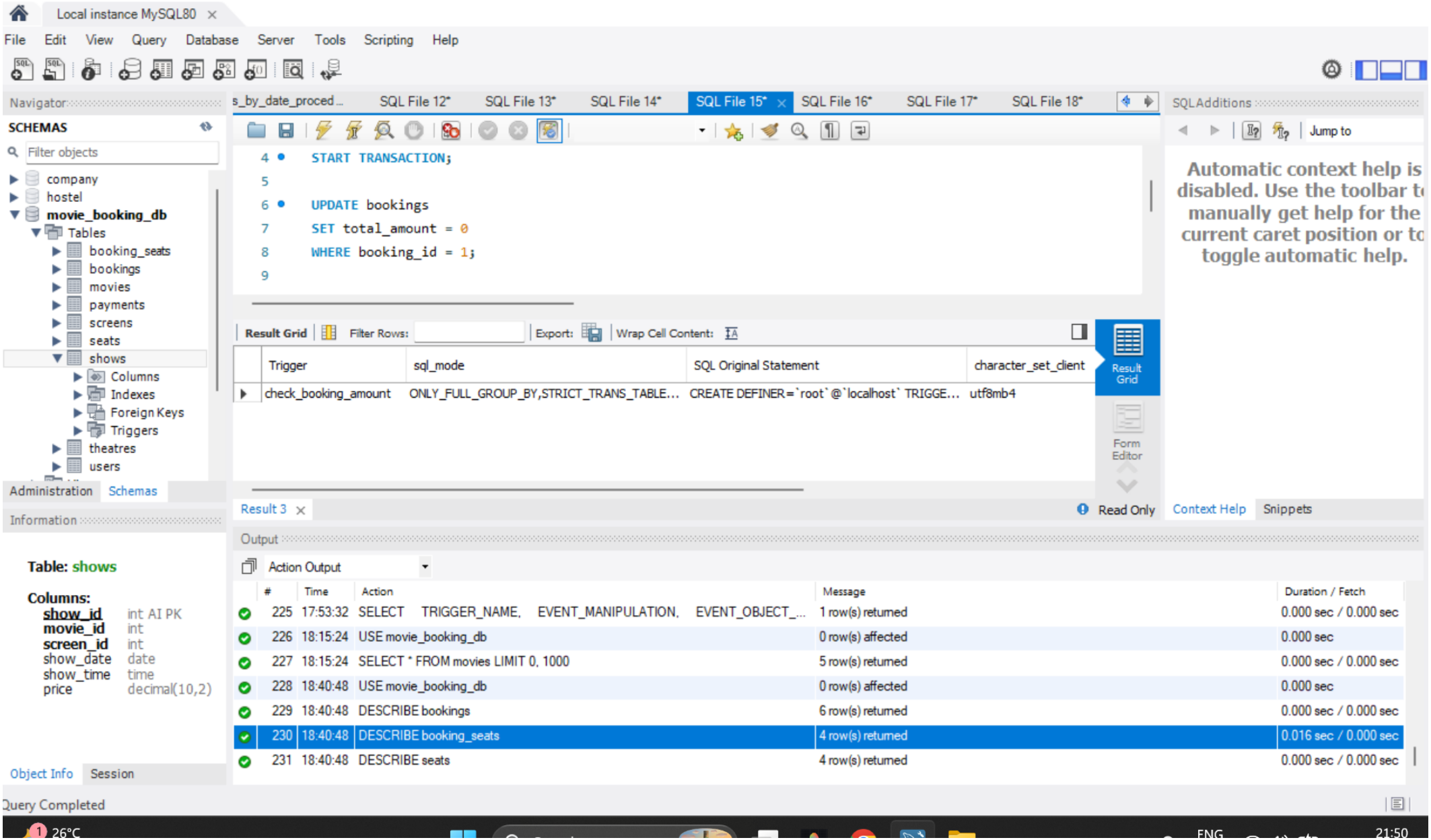
MySQL Workbench Evidence 3

Trigger definition and transaction test context



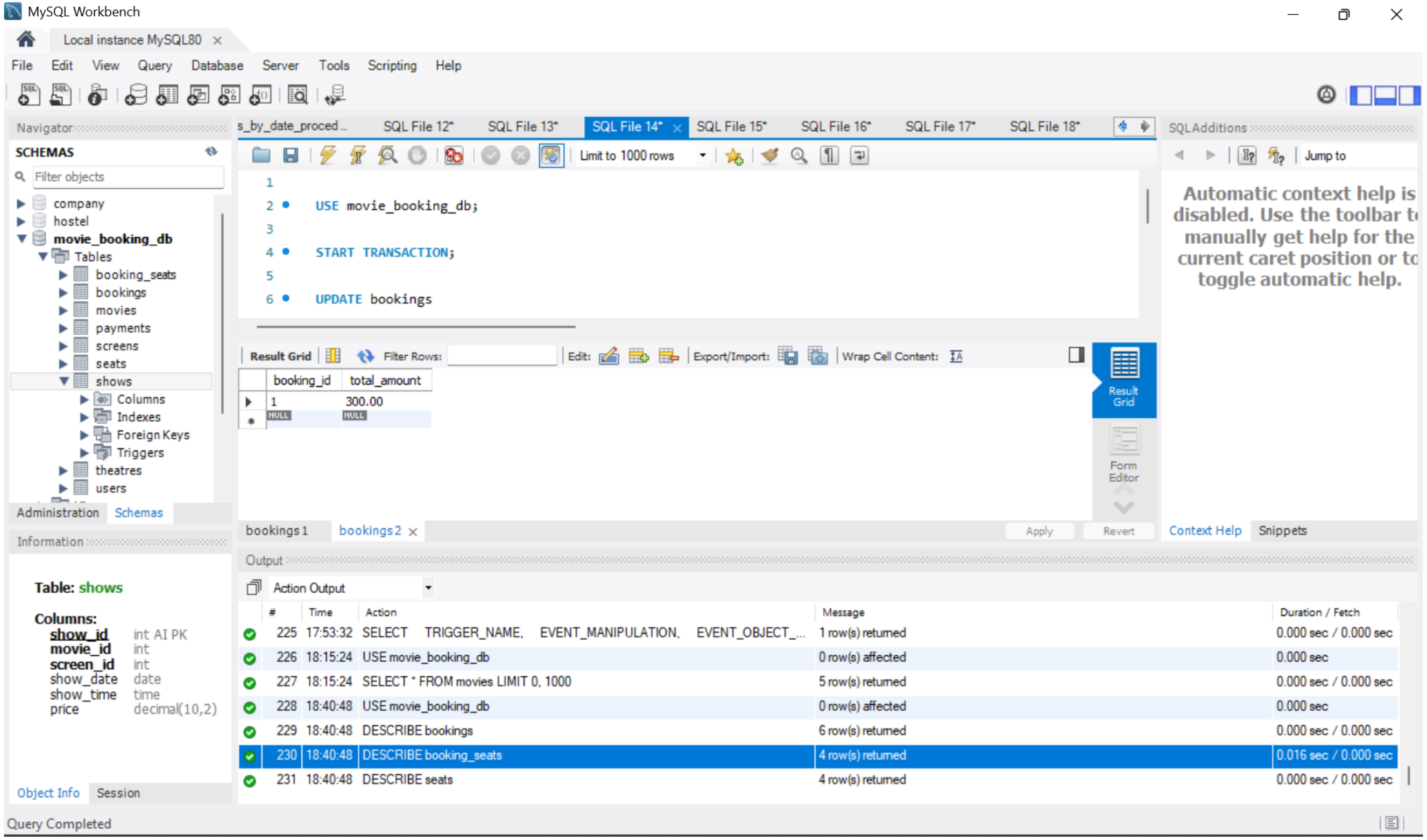
MySQL Workbench Evidence 4

Booking amount validation / transaction test



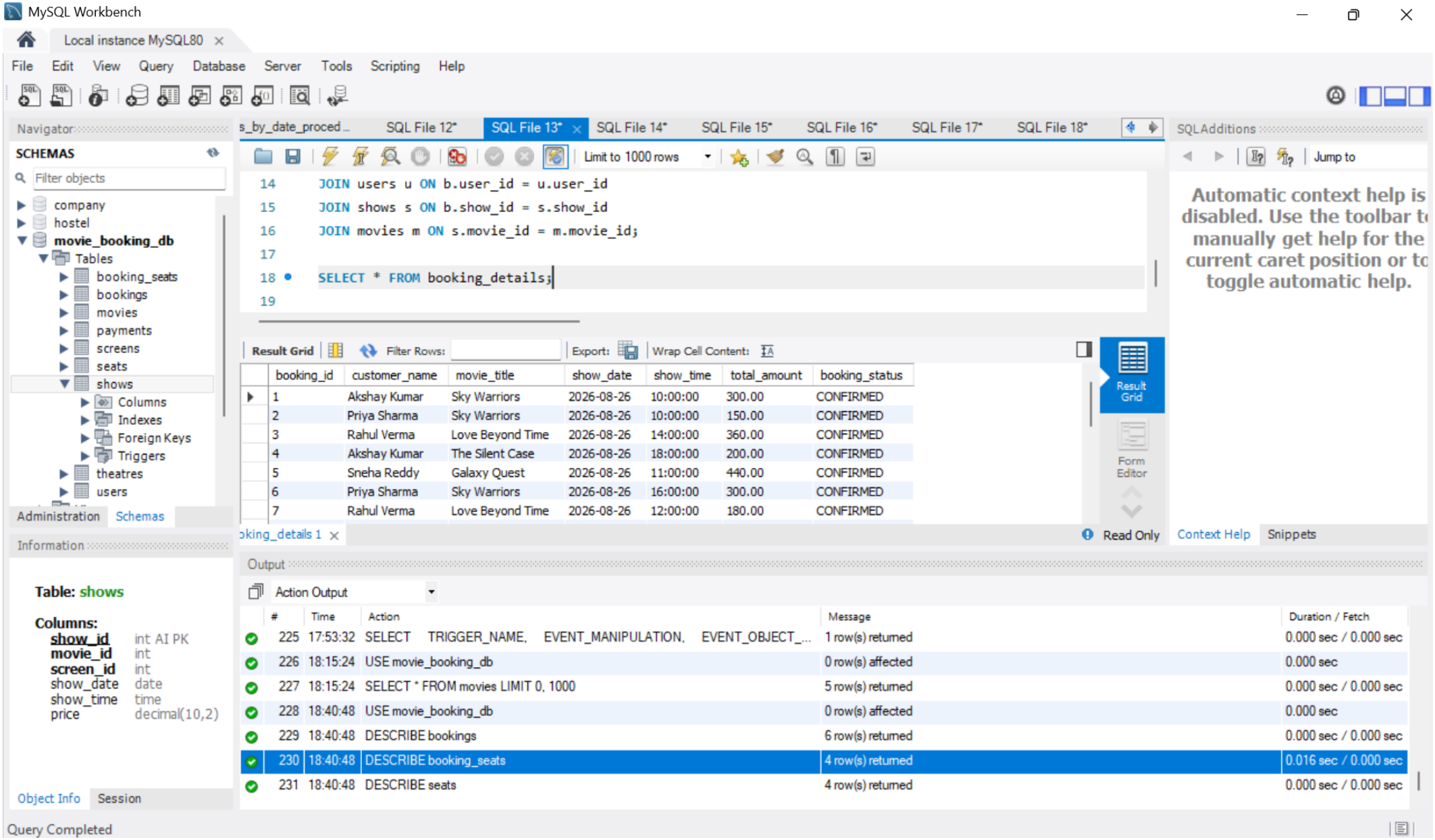
MySQL Workbench Evidence 5

Booking details view: joined customer, movie, show and booking data



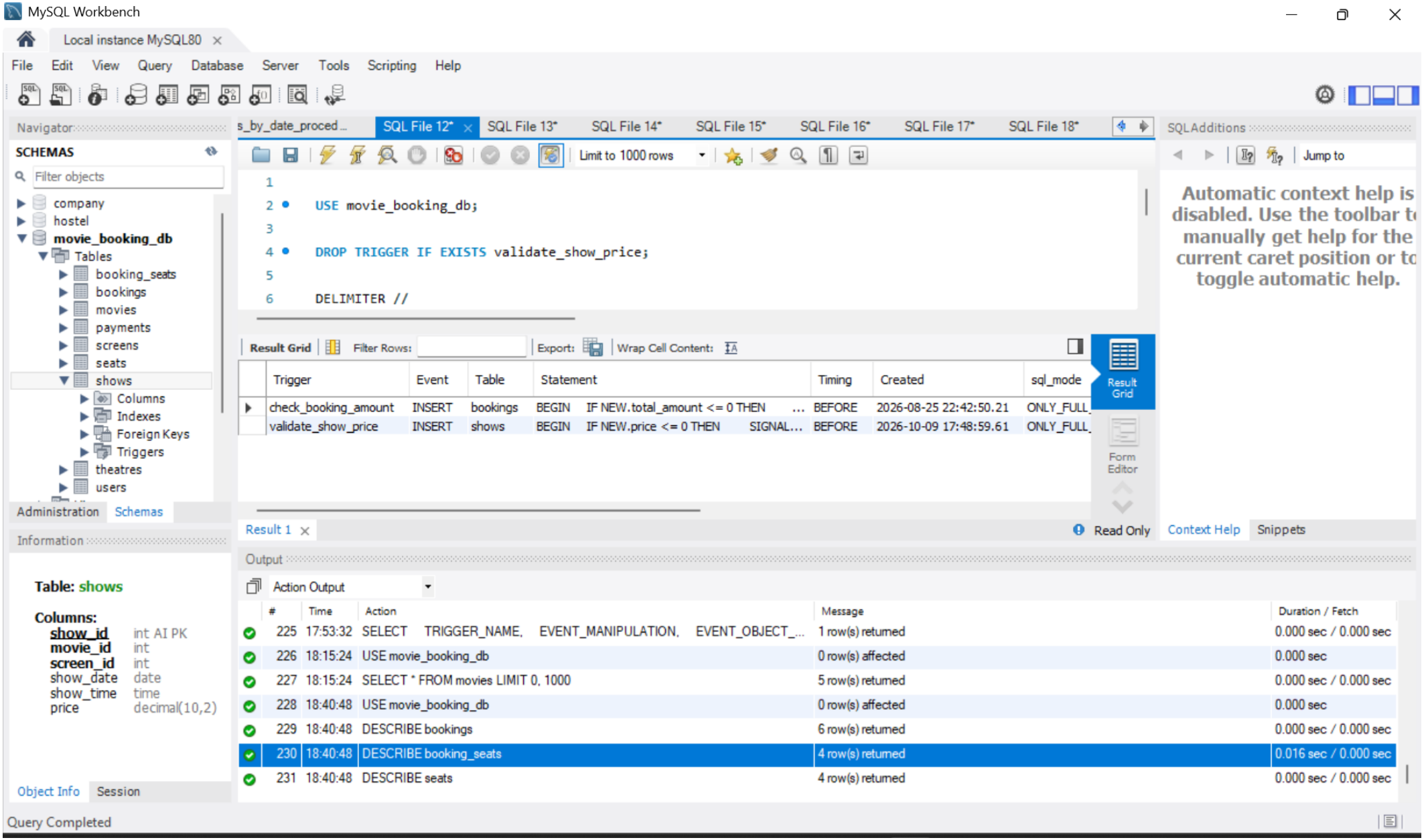
MySQL Workbench Evidence 6

Trigger validation evidence: check_booking_amount and validate_show_price



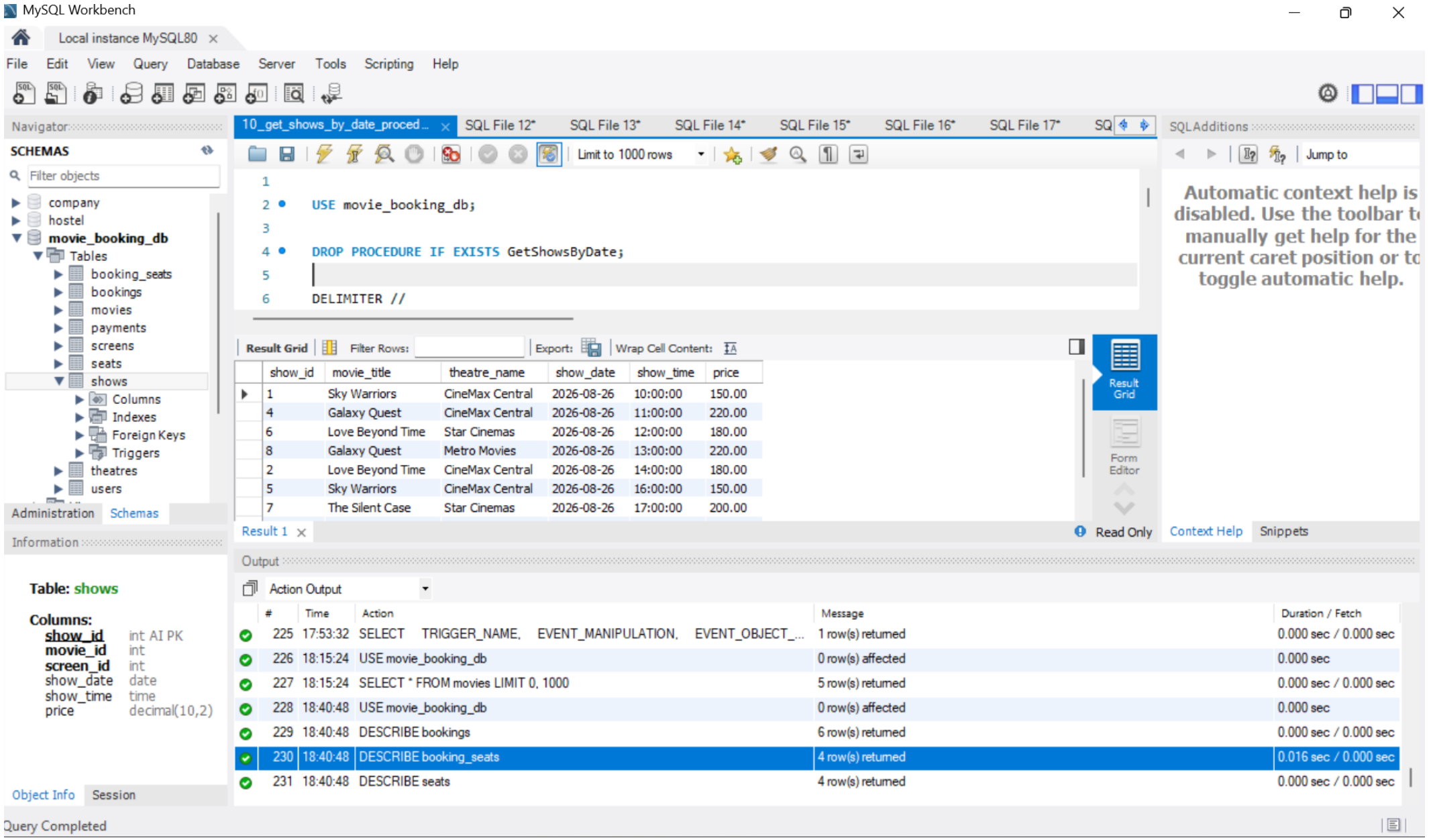
MySQL Workbench Evidence 7

Stored procedure result: GetShowsByDate



MySQL Workbench Evidence 8

SQL object evidence and booking_details query result



Appendix C — Database and SQL Evidence

This appendix adds database evidence documentation to the MovieBook report. The SQL examples below are intended to be run in MySQL Workbench; they are not screenshots of Workbench execution.

C1. Booking records

Shows persisted booking IDs, show IDs, totals, and booking statuses. Earlier test notes reported successful records including booking #39; later website screenshots show bookings #40 and #41.

```
SELECT *
FROM bookings
ORDER BY booking_id DESC
LIMIT 10;
```

C2. Seats associated with bookings

Shows the mapping between booking IDs, show IDs, and seat IDs.

```
SELECT *
FROM booking_seats
ORDER BY booking_seat_id DESC
LIMIT 10;
```

C3. Payment records

Shows payment rows linked to bookings, including the demo payment method and status.

```
SELECT *
FROM payments
ORDER BY payment_id DESC
LIMIT 10;
```

C4. Join and aggregation report

Demonstrates multi-table JOIN, GROUP BY, COUNT, and SUM. Review booking/payment statuses before using totals as formal revenue.

```
SELECT
    m.title AS movie,
    COUNT(b.booking_id) AS total_bookings,
    SUM(b.total_amount) AS total_revenue
FROM movies m
LEFT JOIN shows s ON s.movie_id = m.movie_id
LEFT JOIN bookings b ON b.show_id = s.show_id
GROUP BY m.movie_id, m.title;
```

C5. Stored procedure definition

Use this to display the stored procedure definition if it exists in the selected schema.

```
SHOW CREATE PROCEDURE GetShowsByDate;
```

C6. Trigger inventory

Use this to list triggers in the selected schema. A listing alone does not prove the trigger's behavior; test its expected effect separately.

```
SHOW TRIGGERS;
```

C7. Schema overview

Lists tables available in movie_booking_db.

```
SHOW TABLES;
```

C8. View inventory

Lists views that actually exist in the selected schema.

```
SHOW FULL TABLES WHERE Table_type = 'VIEW';
```

Appendix D — Database structure summary

Table	Role in the application
users	User records referenced by bookings.
movies	Movie titles and descriptive details.
theatres	Cinema/theatre details.
screens	Screens belonging to theatres.
seats	Seat number, screen, and seat type.
shows	Movie schedule, screen, date/time, and ticket price.
bookings	Reservation status, user, show, and total.
booking_seats	Seat allocation per booking and show.
payments	Payment record associated with a booking; demo only.

Reported database safeguards

• Booking creation was reported to use a MySQL transaction, with rollback on failure.

• The API validates seat IDs against the selected show's screen and calculates the total on the server.

• The unique key on (show_id, seat_id) in booking_seats prevents duplicate seat assignment for a show.

• An occupied-seat attempt was reported to return HTTP 409 Conflict.

• Payments are simulated; this project does not process real UPI payments.